

RELATÓRIO FINAL DE INICIAÇÃO CIENTÍFICA

Implementação e Avaliação de Algoritmos de Machine Learning para Previsão de Ataques Cibernéticos

Aluna: Emili Vieira Tabuti

Matrícula: RA00334493

Curso: Ciência da Computação

Grande área CNPq: 1.00.00.00-3 — Ciências Exatas e da Terra

Área CNPq: 1.03.00.00-7 — Ciência da Computação

Orientador: Prof. Dr. Daniel Couto Gatti

Faculdade: Faculdade de Ciências Exatas e Tecnologia — FCET

Projeto do orientador: Sistema de Previsão de Ataques Cibernéticos com Machine Learning para Redes Acadêmicas

Modalidade: PIBIC-CNPq

Período: setembro de 2025 a **TODO: confirmar mês e ano de encerramento**

São Paulo

2026

SUMÁRIO

INTRODUÇÃO	4
PARTE I — ATIVIDADES DESENVOLVIDAS	6
1.1 Sistemática de orientação	6
1.2 Objetivos alcançados	6
1.3 Dificuldades e estratégias	7
1.4 Adequações realizadas durante o desenvolvimento	8
1.5 Atividades acadêmico-científico-culturais	9
PARTE II — RELATÓRIO CIENTÍFICO	10
2.1 Fundamentação teórica	10
2.1.1 Aprendizado de máquina e detecção de intrusões	10
2.1.2 Modelos avaliados	10
2.1.3 Dados, qualidade e validação	11
2.1.4 Métricas	11
2.2 Materiais e método	12
2.2.1 Delineamento	12
2.2.2 Conjuntos de dados	12
2.2.3 Pré-processamento	12
2.2.4 Janelas deslizantes	12
2.2.5 Modelos e hiperparâmetros	13
2.2.6 Validação e rastreamento	13
2.2.7 Implementação e verificação	14
2.2.8 Avaliação da viabilidade prospectiva	14

2.3 Resultados	15
2.3.1 Desenvolvimento temporal e seleção de atributos	15
2.3.2 Avaliação temporal final	15
2.3.3 Desempenho temporal por tipo de ataque	16
2.3.4 Serviço de classificação	16
2.3.5 Resultados da avaliação prospectiva	16
2.4 Discussão	17
2.5 Avaliação dos procedimentos	18
2.6 Conclusões e trabalhos futuros	19
PARTE III — RESUMO ESTENDIDO	21
Introdução	21
Objetivos	22
Metodologia	22
Resultados e discussão	23
Conclusões	24
Agência de Fomento	24
Referências	25
REFERÊNCIAS BIBLIOGRÁFICAS	26
ASSINATURAS	27

INTRODUÇÃO

Este relatório apresenta os resultados finais do plano individual de Emili Vieira Tabuti, desenvolvido no Programa Institucional de Bolsas de Iniciação Científica da PUC-SP. A pesquisa investigou algoritmos de aprendizado de máquina aplicados à segurança de redes, com ênfase na implementação, na avaliação e na disponibilização dos modelos.

O projeto do orientador propôs um sistema de previsão de ataques para redes acadêmicas, articulando processamento de tráfego, aplicação de modelos computacionais e visualização de alertas. Nesse contexto, o plano individual reuniu atividades de revisão bibliográfica, implementação, treinamento, comparação, integração, testes e divulgação científica.

O desenvolvimento utilizou o conjunto público UNSW-NB15. Foram comparados Random Forest, Decision Tree e uma rede Long Short-Term Memory mediante holdout cronológico por sessões naturais e folds temporais expansivos com purga. A avaliação considerou F1, PR-AUC, AUC-ROC, precisão, revocação e taxa de falsos positivos.

Os registros temporais do UNSW-NB15 foram analisados para verificar se permitiam formular a previsão antecipada. Foram definidos horizontes de 5, 15, 30 e 60 segundos, com ordenação temporal dos fluxos e uso exclusivo das informações disponíveis em cada instante. O protocolo também excluiu períodos com ataques ativos ou recentes. Após essa filtragem, permaneceram cinco eventos positivos estritos, quantidade que não permitiu formar conjuntos de treino, validação e teste com diversidade suficiente para avaliar a antecipação de novos ataques.

Diante das características encontradas na base, o trabalho prosseguiu com a implementação e a avaliação da detecção de ataques a partir do histórico recente do tráfego. Essa tarefa é compatível com os rótulos disponíveis e permitiu comparar os modelos, construir os artefatos e integrar a solução ao painel.

O pipeline organizou os registros em janelas deslizantes de tamanho dez, sempre isoladas por partição, sessão e arquivo-fonte. O pré-processamento e a seleção de atributos foram ajustados somente no trecho de treino de cada fold. Foram comparadas as variantes com todas as variáveis e com as 10, 20 e 30 mais importantes. O protocolo congelou Decision Tree com 10 atributos, LSTM com 20 e Random Forest com 30 antes de qualquer abertura do teste futuro.

Depois do ajuste final em treino e validação, os três modelos foram avaliados juntos uma única vez em 306.701 janelas da sessão futura. O Random Forest permaneceu vencedor, com F1 de 0,9261, PR-AUC de 0,9856 e revocação de 0,9525. Seu F1 diferiu em apenas 0,0025 da média obtida no desenvolvimento, evidência de estabilidade entre os folds temporais e o período fechado.

O modelo vencedor foi empacotado com o pré-processador e o ranking de 30 atributos em um artefato consumido pela API REST integrada ao painel. Nesse desenho, o rótulo corresponde ao último registro da janela, enquanto os registros anteriores ajudam a identificar o estado atual do

tráfego. A previsão antecipada permanece como continuidade e requer mais eventos de ataque separados, precedidos por períodos benignos.

PARTE I — ATIVIDADES DESENVOLVIDAS

1.1 Sistemática de orientação

No primeiro semestre, a orientação ocorreu por reuniões quinzenais com o Prof. Dr. Daniel Couto Gatti. As reuniões acompanharam entregas, dependências e ajustes técnicos.

A equipe utilizou documentação compartilhada e controle de versão. Esses recursos registraram decisões, histórias de implementação e resultados experimentais.

Os artefatos de planejamento organizaram o trabalho em três frentes: preparação dos dados, treinamento dos modelos e visualização dos resultados.

Caroline Guimarães Campos trabalhou na preparação e no processamento dos dados. Emili Vieira Tabuti concentrou-se na implementação, no treinamento e na avaliação dos modelos. Isabela Groke Gomes foi responsável pelo desenvolvimento do painel e pela visualização dos resultados.

TODO: confirmar se a periodicidade quinzenal foi mantida após março de 2026.

1.2 Objetivos alcançados

A revisão bibliográfica analisou quatorze trabalhos publicados entre 2017 e 2025. O levantamento reuniu estudos experimentais, revisões e propostas de avaliação.

Essa revisão apoiou a escolha de Random Forest, Decision Tree e LSTM. Também fundamentou o uso de F1, AUC-ROC, precisão, revocação e FPR.

O ambiente de desenvolvimento foi estruturado em Python, com as dependências necessárias registradas para permitir a reprodução das execuções.

O pipeline passou a carregar e validar o UNSW-NB15 não escalonado em formato Parquet, preservando timestamps, arquivo-fonte e identificadores.

O módulo de engenharia de atributos criou janelas deslizantes. Ele gerou representações sequenciais para LSTM e tabulares para modelos baseados em árvores.

Foi implementado e testado um seletor de atributos baseado em Random Forest, com opções de seleção por quantidade ou limiar de importância. No protocolo temporal, o seletor foi integrado ao caminho real de treinamento: ele aprende o ranking apenas no treino de cada fold e transforma a validação sem refit.

Foram executadas 36 combinações de desenvolvimento, formadas por três modelos, quatro quantidades de atributos e três folds temporais. O protocolo congelado selecionou as variantes `top_10` para Decision Tree, `top_20` para LSTM e `top_30` para Random Forest.

Os três modelos finais foram ajustados em treino+validação e avaliados na mesma sessão futura. Cada execução registrou métricas, matrizes de confusão, previsões por janela, tempos, dimensões e hashes dos artefatos.

Os resultados experimentais foram preservados em JSON e CSV, com manifestos e hashes SHA-256 para rastreabilidade.

A avaliação incluiu desempenho global e análise por tipo de ataque. Essa etapa revelou diferenças que a métrica binária agregada ocultava.

O Random Forest temporal vencedor foi empacotado em um artefato portátil que contém o estado do pré-processador ajustado em treino+validação, os 30 atributos selecionados, a janela, o limiar, os rótulos e os hashes do protocolo e das métricas finais.

Uma API FastAPI disponibilizou predição, saúde, metadados e histórico. O serviço carrega o Random Forest por padrão.

O contrato REST permitiu integração com o painel. Também foi criado um endpoint simulado para desenvolvimento paralelo.

A versão auditada apresenta 175 testes aprovados. A suíte também cobre separação temporal, purga, pré-processamento por fold, seleção integrada, janelas isoladas, congelamento do protocolo, avaliação final e inferência pela API.

1.3 Dificuldades e estratégias

O treinamento da LSTM exigiu memória e capacidade computacional elevadas. O volume aumentou porque cada janela replica dez registros.

O código passou a usar números de 32 bits. Essa alteração reduziu aproximadamente pela metade a memória dos arrays.

O treinamento também adotou lotes gerados sob demanda. A estratégia evitou copiar partições completas para a memória.

As árvores apresentaram consumo elevado no UNSW-NB15. O pré-processamento temporal gera 204 atributos, e cada janela com todos eles produziria 2.040 valores.

O número de processos paralelos foi limitado a dois. A profundidade máxima das árvores foi fixada em vinte.

Essas decisões priorizaram estabilidade computacional. Elas alteraram os hiperparâmetros preliminares, que não limitavam a profundidade.

O UNSW-NB15 exige conversão numérica e codificação de variáveis categóricas. O pipeline incorporou essas transformações dentro de cada fold.

Os treinamentos foram executados localmente. A seleção reduziu a entrada para 10, 20 ou 30 atributos, as janelas foram materializadas em lotes e o ambiente `.venv-tf` continha TensorFlow. A LSTM final foi o modelo Keras real.

A auditoria posterior identificou que o ambiente temporal efetivo possuía versões de NumPy, Pandas, PyArrow e scikit-learn diferentes das registradas no congelamento a partir do ambiente padrão. O desvio foi documentado com o hash do executor; nenhuma configuração foi alterada e o teste não foi repetido.

A integração do painel exigiu uma API antes dos artefatos finais. O endpoint simulado preservou o contrato durante esse intervalo.

Na versão integrada, a API passou a classificar o tráfego com o modelo treinado. O serviço valida atributos, tamanho da janela e valores numéricos.

1.4 Adequações realizadas durante o desenvolvimento

O plano de trabalho previa o uso de dados da universidade ou de bases públicas. Para garantir disponibilidade, metadados temporais e reprodução, foi utilizado o UNSW-NB15.

A revisão bibliográfica orientou a escolha de Random Forest, Decision Tree e LSTM. Esses modelos permitiram comparar abordagens baseadas em árvores com uma rede neural voltada ao processamento de sequências.

O módulo de seleção de atributos foi incorporado aos experimentos finais. Cada fold reajustou o ranking somente em seu treino, e cada modelo foi avaliado com todas as variáveis e com `top_n` igual a 10, 20 ou 30.

Os experimentos utilizaram janelas de dez registros, três folds cronológicos expansivos com purga e uma sessão final posterior mantida fechada. Essa adequação corrigiu a sobreposição entre períodos e passou a medir generalização temporal no UNSW-NB15.

A pesquisa também analisou a possibilidade de prever ataques futuros com o UNSW-NB15. Foram definidos horizontes temporais e utilizados apenas dados disponíveis antes do instante analisado.

Após a separação de períodos com ataques ativos ou recentes, permaneceram cinco eventos adequados ao protocolo de previsão. Essa quantidade não permitiu formar conjuntos suficientemente diversos para treinar e avaliar um modelo de antecipação.

Diante dessa característica da base, a implementação prosseguiu com a detecção de ataques a partir do histórico recente do tráfego. Essa abordagem permitiu concluir a comparação dos modelos, gerar os artefatos e integrar a solução à API e ao painel.

A previsão antecipada permanece como continuidade da pesquisa. Sua avaliação dependerá de uma nova base com mais eventos de ataque separados, períodos benignos anteriores e registros temporais bem identificados.

O sistema também foi utilizado em um cenário demonstrativo de SYN flood. Esse cenário permitiu verificar a comunicação entre o modelo, a API e o painel, complementando os testes automatizados do projeto.

1.5 Atividades acadêmico-científico-culturais

No primeiro período, a estudante concluiu o curso Advanced Solutions Lab for AI & ML, da Google Cloud. O curso abordou redes neurais e engenharia de atributos.

TODO: informar seminários, cursos, apresentações ou eventos realizados após março de 2026.

PARTE II — RELATÓRIO CIENTÍFICO

2.1 Fundamentação teórica

2.1.1 Aprendizado de máquina e detecção de intrusões

Os sistemas tradicionais de detecção de intrusões reconhecem assinaturas, regras ou desvios previamente definidos. Embora sejam importantes para identificar ameaças conhecidas, essas abordagens exigem atualizações frequentes. O aprendizado de máquina pode complementar esse processo ao encontrar padrões nos dados de tráfego e apoiar a identificação de comportamentos incomuns.

Bertoli et al. (2021) apresentam um fluxo completo para detecção de intrusões, desde a coleta e a preparação dos atributos até o uso do modelo. Os autores também relacionam desempenho e custo computacional, mostrando que a escolha de um algoritmo deve considerar tanto as métricas quanto sua aplicação prática. Halbouni et al. (2022) ampliam essa discussão ao reunir diferentes técnicas de aprendizado tradicional e profundo utilizadas na segurança cibernética.

Ankalaki et al. (2025) tratam a previsão de ataques como uma área que envolve aprendizado tradicional, redes profundas e métodos de explicação dos resultados. Nesse contexto, é importante diferenciar duas tarefas. A detecção identifica um ataque presente nos dados analisados, enquanto a previsão procura estimar um evento antes que ele aconteça. Para avaliar previsão antecipada, é necessário definir um intervalo futuro e construir rótulos que representem esse período.

2.1.2 Modelos avaliados

Este trabalho avaliou Decision Tree, Random Forest e LSTM por representarem abordagens diferentes de aprendizado supervisionado. A Decision Tree organiza decisões sucessivas a partir dos atributos de entrada, formando uma estrutura que pode ser acompanhada e interpretada. Essa característica facilita a análise do comportamento do modelo, embora árvores muito complexas possam se ajustar excessivamente aos dados. Bertoli et al. (2021) também destacam sua eficiência em aplicações de detecção de intrusões.

O Random Forest combina várias árvores treinadas com diferentes amostras e subconjuntos de atributos. A decisão final resulta da combinação dessas árvores, o que reduz a dependência de uma única estrutura e tende a produzir resultados mais estáveis. Chakir et al. (2023) observaram desempenho competitivo de métodos de ensemble na identificação de ataques web, reforçando sua aplicação em problemas de segurança.

A LSTM pertence ao grupo das redes neurais recorrentes e foi desenvolvida para trabalhar com informações organizadas em sequência. Seus mecanismos de memória permitem conservar ou descartar informações ao longo dos dados analisados. Yin et al. (2017) aplicaram redes recorrentes à detecção de intrusões e obtiveram desempenho superior ao de diferentes classificadores no NSL-KDD.

Berman et al. (2019) observam que dados de segurança podem apresentar relações temporais, o que justifica a investigação de redes recorrentes nesse domínio. Em contrapartida, o treinamento da LSTM costuma exigir mais memória, tempo de processamento e ajuste de parâmetros do que os modelos baseados em árvores.

2.1.3 Dados, qualidade e validação

O desempenho de um modelo depende não apenas do algoritmo escolhido, mas também da qualidade e da organização dos dados. Duplicações, sobreposições e rótulos inconsistentes podem alterar as métricas e dificultar a análise dos resultados. Tran et al. (2022) mostram que registros duplicados influenciam a avaliação de sistemas de detecção e recomendam uma etapa de preparação cuidadosa antes da comparação dos modelos.

Outro aspecto importante é a variedade de protocolos utilizados nas pesquisas. Le Jeune, Goedemé e Mentens (2021) observam que estudos da área adotam diferentes conjuntos, atributos, divisões e métricas, o que dificulta comparações diretas. Os autores defendem procedimentos mais uniformes e descrições completas das etapas experimentais.

A validação cruzada estratificada contribui para essa padronização ao preservar a proporção das classes em cada partição e reduzir a dependência de uma única divisão dos dados. Ainda assim, é necessário considerar a relação entre os exemplos. Registros duplicados ou janelas que compartilham observações podem gerar amostras muito semelhantes em partições diferentes.

Essa preocupação se torna ainda mais importante em tarefas temporais. Uma divisão aleatória pode distribuir registros do passado e do futuro entre treino e validação. Por isso, estudos voltados à previsão devem preservar a ordem temporal e separar adequadamente arquivos, períodos ou eventos de ataque.

2.1.4 Métricas

Em conjuntos desbalanceados, a acurácia isolada pode oferecer uma visão incompleta do desempenho. Um modelo que favorece a classe mais frequente pode alcançar acurácia elevada mesmo sem identificar adequadamente os ataques. Por esse motivo, a avaliação utilizou métricas complementares.

A precisão representa a proporção de alertas positivos que realmente correspondem a ataques, enquanto a revocação indica quantos ataques existentes foram identificados. Essas duas medidas ajudam a observar, respectivamente, a ocorrência de alarmes falsos e a quantidade de ataques não detectados. O F1 combina precisão e revocação em uma única medida e permite analisar o equilíbrio entre elas.

A taxa de falsos positivos, ou FPR, mede a proporção de registros benignos classificados como ataque. Esse valor possui importância operacional, pois muitos alarmes incorretos aumentam o

trabalho de análise. A AUC-ROC, por sua vez, resume a capacidade do modelo de separar as classes em diferentes limiares, complementando as métricas calculadas para a classificação final.

2.2 Materiais e método

2.2.1 Delineamento

A pesquisa adotou abordagem quantitativa e experimental. Foram comparados três algoritmos no UNSW-NB15 com avaliação temporal.

O objetivo empírico foi medir detecção binária de tráfego benigno e malicioso em dados posteriores aos usados no desenvolvimento. A unidade de avaliação foi uma janela de dez registros, cujo alvo corresponde ao último registro.

Não houve participantes humanos, entrevistas ou intervenção clínica. A pesquisa analisou dados públicos de tráfego previamente coletado.

2.2.2 Conjuntos de dados

O UNSW-NB15 limpo contém 1.584.259 registros, 43 campos brutos elegíveis e 204 atributos após o pré-processamento ajustado no treino. Existem 1.523.904 registros benignos e 60.355 ataques.

As categorias incluem Exploits, Generic, Fuzzer, Reconnaissance, DoS, Analysis, Backdoor, Shellcode e Worms.

Os dados foram armazenados em Parquet. O pipeline manteve `Binary_Label` como alvo e `Attack_Type` para análise granular.

2.2.3 Pré-processamento

A limpeza removeu duplicações exatas, tratou valores ausentes e substituiu infinitos. Os nomes das colunas foram padronizados.

Variáveis numéricas assimétricas receberam transformação logarítmica. Depois, o `RobustScaler` utilizou mediana e intervalo interquartil.

O UNSW-NB15 também recebeu codificação one-hot para protocolo, estado e serviço. O processamento temporal resultou em 204 atributos. Timestamps, IPs, rótulos, sessão, partição e identificadores permaneceram apenas como metadados.

Em cada fold, transformação logarítmica, categorias e estatísticas do `RobustScaler` foram aprendidas somente no treino. A validação recebeu apenas `transform`. No ajuste final, pré-processador e seletor usaram treino+validação antes da única abertura do teste.

2.2.4 Janelas deslizantes

Cada amostra contém dez registros consecutivos. O alvo corresponde ao rótulo do décimo registro.

A LSTM recebeu um tensor tridimensional. Random Forest e Decision Tree receberam a mesma janela achatada.

O procedimento preservou o contexto dos nove registros anteriores. Entretanto, ele não deslocou o alvo para um instante futuro.

As janelas foram construídas com passo unitário. Assim, duas janelas consecutivas compartilham nove registros dentro da mesma partição, mas nenhuma janela cruza partição, sessão, bloco de desenvolvimento ou arquivo-fonte.

2.2.5 Modelos e hiperparâmetros

O Random Forest utilizou 100 árvores, profundidade máxima igual a 20 e dois processos paralelos. A raiz aleatória foi 42.

A Decision Tree utilizou critério Gini e profundidade máxima igual a 20. O mínimo para divisão foi duas amostras.

A LSTM utilizou 64 unidades recorrentes e uma camada densa com 32 unidades. A saída binária aplicou função sigmoide.

O treinamento da LSTM executou dez épocas, lotes de 4.096 amostras, `shuffle=False` e otimizador Adam.

2.2.6 Validação e rastreamento

O UNSW-NB15 foi ordenado de forma estável por `Stime`, `Ltime`, arquivo-fonte e `record_id`. Duas lacunas superiores a uma hora delimitaram três sessões naturais. Após purgar nove registros de cada lado das fronteiras, foram destinados 1.023.187 registros ao treino, 254.326 à validação e 306.710 ao teste fechado.

O desenvolvimento utilizou três folds cronológicos expansivos, sem embaralhamento. As fronteiras aplicaram purga mínima de nove registros e embargo adicional quando necessário para garantir que o último `Ltime` do treino fosse anterior ao primeiro `Stime` da validação.

Em cada fold foram avaliados os três algoritmos com todos os atributos e com os 10, 20 e 30 primeiros do ranking aprendido no próprio treino, totalizando 36 execuções. A métrica primária pré-definida foi o F1 médio; em seguida seriam considerados menor FPR e menor número de atributos.

O protocolo foi congelado antes do teste. Decision Tree `top_10`, Random Forest `top_30` e LSTM `top_20` foram ajustados em treino+validação e avaliados juntos uma única vez em 306.701 janelas futuras. O teste não participou de seleção, limiar ou hiperparâmetros.

F1, PR-AUC, AUC-ROC, precisão, revocação e FPR foram preservados em JSON e CSV, junto com matrizes de confusão, previsões, tempos e resultados por tipo de ataque. Hashes registram dados, código, protocolo e artefatos.

2.2.7 Implementação e verificação

O código foi organizado em módulos de dados, atributos, treinamento, modelos e API. Essa separação favoreceu testes e manutenção.

O protocolo temporal foi executado localmente. A redução de atributos e o processamento em lotes permitiram treinar a LSTM TensorFlow real em CPU, sem necessidade de Google Colab nessa etapa.

A suíte automatizada executou 175 testes com sucesso. Um aviso conhecido de compatibilidade entre dependências não impediu a aprovação.

O Random Forest vencedor foi serializado com modelo, pré-processador, ranking `top_30`, janela, limiar, esquema bruto, rótulos e hashes. A API verifica esses metadados no carregamento e aplica internamente todo o caminho de transformação.

2.2.8 Avaliação da viabilidade prospectiva

A etapa prospectiva utilizou os campos `Stime` e `Ltime` do UNSW-NB15 antes do escalonamento. Os 1.584.259 fluxos foram ordenados por arquivo de origem, instante inicial e instante final, reduzindo de 250.850 para zero as regressões temporais presentes na ordem original.

Considerou-se um ataque ativo sempre que algum fluxo malicioso cobrisse o instante analisado no intervalo inclusivo `[Stime, Ltime]`. Essa definição incorporou a duração dos fluxos e evitou interpretar como novos eventos os instantes intermediários de um ataque já iniciado.

Os rótulos prospectivos indicaram se um ataque começaria nos 5, 15, 30 ou 60 segundos seguintes. Instantes com ataque em andamento foram excluídos, e os sessenta atributos históricos foram calculados apenas com fluxos concluídos nos 30, 60 ou 120 segundos anteriores.

Para distinguir a antecipação de um novo ataque da simples recorrência, o conjunto estrito também removeu instantes que continham ataques concluídos nos 120 segundos anteriores. A divisão preservou a ordem dos eventos, aplicou uma purga de 180 segundos e manteve o conjunto de teste indisponível para seleção de modelos ou limiares.

Como os dados não ofereceram eventos suficientes para uma avaliação antecipada completa, os módulos dessa etapa não foram conectados à API nem ao painel. O protótipo operacional prosseguiu com a detecção de ataques a partir do histórico recente do tráfego.

2.3 Resultados

2.3.1 Desenvolvimento temporal e seleção de atributos

Os 36 experimentos de desenvolvimento compararam todas as variáveis com os recortes `top_10`, `top_20` e `top_30`. A Tabela 1 apresenta a melhor variante de cada algoritmo segundo o F1 médio nos três folds cronológicos.

Tabela 1 — Configurações selecionadas no desenvolvimento temporal

Modelo	Variante	F1 médio $\pm$ desvio	PR-AUC	AUC-ROC	FPR
Decision Tree	<code>top_10</code>	0,8923 $\pm$ 0,0235	0,8037	0,9507	0,0070
LSTM	<code>top_20</code>	0,8812 $\pm$ 0,0156	0,9138	0,9954	0,0129
Random Forest	<code>top_30</code>	0,9236 $\pm$ 0,0211	0,9728	0,9985	0,0084

Fonte: `ml-pipeline/reports_temporal/unsw/development_experiments/comparison_metrics.csv` (2026).

A seleção foi utilizada efetivamente nos modelos. Em comparação com a variante de 204 atributos, o F1 médio passou de 0,8797 para 0,8923 na Decision Tree, de 0,8779 para 0,8812 na LSTM e de 0,9179 para 0,9236 no Random Forest. No vencedor, a entrada por registro foi reduzida em 85,3%, de 204 para 30 atributos.

2.3.2 Avaliação temporal final

O protocolo congelou as três configurações antes do teste. Depois do ajuste em 1.277.513 registros de treino+validação, os modelos foram avaliados juntos nas mesmas 306.701 janelas da sessão futura. A Tabela 2 apresenta a única avaliação final autorizada.

Tabela 2 — Desempenho no teste cronológico fechado do UNSW-NB15

Modelo	Atributos	F1	PR-AUC	AUC-ROC	Precisão	Revocação	FPR
Decision Tree	10	0,8898	0,8231	0,9569	0,8789	0,9010	0,0111
LSTM	20	0,8846	0,9415	0,9954	0,8280	0,9494	0,0177
Random Forest	30	0,9261	0,9856	0,9987	0,9010	0,9525	0,0094

Fonte: `ml-pipeline/reports_temporal/unsw/final_test_metrics.json` (2026).

O Random Forest, escolhido no desenvolvimento, permaneceu vencedor no período futuro. Seu F1 no teste foi 0,0025 maior que a média de desenvolvimento. As diferenças foram $-0,0025$ para Decision Tree e $+0,0034$ para LSTM, sem qualquer reescolha ou ajuste posterior.

As matrizes de confusão (TN, FP, FN, TP) foram 278.310, 3.134, 2.501 e 22.756 para Decision Tree; 276.462, 4.982, 1.277 e 23.980 para LSTM; e 278.802, 2.642, 1.199 e 24.058 para Random Forest.

2.3.3 Desempenho temporal por tipo de ataque

A Tabela 3 segmenta as previsões binárias do teste futuro por categoria. Ela não representa uma classificação multiclasse, pois cada categoria é comparada com os registros benignos.

Tabela 3 — F1 por tipo de ataque no teste temporal do UNSW-NB15

Tipo de ataque	Positivos	Decision Tree	LSTM	Random Forest
Analysis	621	0,2592	0,1923	0,3054
Backdoor	623	0,2778	0,1963	0,3187
DoS	1.465	0,4502	0,3602	0,5224
Exploits	6.851	0,7725	0,7089	0,8336
Fuzzer	4.970	0,6036	0,6190	0,6776
Generic	6.894	0,8069	0,7255	0,8387
Reconnaissance	3.420	0,6749	0,5573	0,7208
Shellcode	371	0,1630	0,1241	0,2187
Worms	42	0,0230	0,0158	0,0301

Fonte: ml-pipeline/reports_temporal/unsu/tables/attack_type_metrics.csv (2026).

O Random Forest obteve o maior F1 em todas as categorias, mas a precisão permaneceu baixa nas classes raras. Em Worms, por exemplo, a revocação foi 0,9762, porém a precisão de 0,0153 limitou o F1 a 0,0301. O resultado mostra por que métricas globais e por categoria devem ser apresentadas em conjunto.

2.3.4 Serviço de classificação

O artefato temporal do Random Forest ocupa aproximadamente 38 MB e recebe dez registros com 43 campos brutos. Internamente, ele aplica o pré-processamento, seleciona os 30 atributos fixados, forma 300 valores por janela e executa a classificação.

A API carrega esse artefato por padrão. `POST /predict` valida o esquema e retorna classe e confiança; `GET /health` e `GET /model/info` expõem estado, versão, entradas e atributos selecionados; `GET /history` mantém até cem previsões em memória.

A validação usou dez registros de treino, nunca o teste. A resposta HTTP coincidiu exatamente com a inferência direta (BENIGN, confiança 0,9999224673), uma entrada incompleta retornou erro 422 e o modelo carregou em 0,155 segundo no ambiente compatível.

2.3.5 Resultados da avaliação prospectiva

A auditoria temporal identificou 66.484 instantes distintos e 277 períodos de ataque após considerar a duração dos fluxos. Dos 28.625 instantes cobertos por ataques ativos, 28.623 também continham tráfego benigno, evidenciando forte sobreposição entre os dois estados.

Antes do filtro estrito, o número de rótulos positivos variou entre 337 e 466, conforme o horizonte, com antecedência mediana de apenas um segundo. Contudo, entre 85% e 97% desses casos possuíam tráfego malicioso concluído no histórico. A separação aparente refletia, assim, principalmente persistência ou recorrência de ataques, e não sinais anteriores ao primeiro ataque recente.

Depois da remoção desses históricos, permaneceram 37.311 instantes e somente cinco eventos positivos estritos. A Tabela 5 apresenta a distribuição das classes nos quatro horizontes.

Tabela 5 — Distribuição estrita da avaliação prospectiva

Horizonte	Positivos	Negativos	Condição nas partições futuras
5 s	10	37.301	ambas as classes; um evento na validação e um no teste
15 s	24	37.287	ambas as classes; um evento na validação e um no teste
30 s	43	37.268	sem negativos na validação e no teste
60 s	46	37.265	sem negativos na validação e no teste

Fonte: artefatos em `ml-pipeline/reports_local/prospective/` (2026).

O treino recebeu três eventos, enquanto a validação e o teste receberam apenas um evento cada. Essa distribuição não permite estimar de forma estável a capacidade de generalização, pois um único evento pode alterar substancialmente qualquer métrica.

Nos horizontes de 30 e 60 segundos, as partições futuras também não apresentaram exemplos negativos. Esse perfil dos dados não ofereceu base suficiente para escolher um modelo prospectivo final, e o teste foi preservado sem uso na seleção de atributos, limiares ou algoritmos.

2.4 Discussão

O Random Forest apresentou F1 de 0,9261 na sessão futura. O período de teste é posterior, não compartilha registros com o desenvolvimento e todas as transformações foram ajustadas sem acesso a ele. Diferenças nas divisões e nas métricas podem modificar substancialmente as conclusões (LE JEUNE; GOEDEMÉ; MENTENS, 2021).

A proximidade entre desenvolvimento e teste temporal fortalece a interpretação. O F1 do Random Forest passou de 0,9236 para 0,9261, enquanto Decision Tree e LSTM variaram menos de 0,0034 em valor absoluto.

A seleção de atributos integrou o resultado científico. As três variantes escolhidas superaram as configurações com todas as 204 variáveis no F1 médio, além de reduzir memória e custo de treinamento. O Random Forest `top_30` venceu pela regra previamente definida, embora sua

diferença para `top_20` tenha sido de apenas 0,0000299 no desenvolvimento. Como nenhuma tolerância de empate havia sido pré-registrada, a decisão não foi modificada posteriormente.

O Random Forest ofereceu o melhor compromisso de F1, PR-AUC, precisão, revocação e FPR no teste futuro, além de produzir um artefato de integração direta com a API. Esses fatores justificaram sua escolha para o protótipo, mas não demonstram superioridade universal sobre os demais modelos.

A análise por categoria revelou fragilidades que as métricas globais ocultam. Analysis, Backdoor, Shellcode e Worms apresentaram F1 reduzido mesmo no modelo vencedor. Como destacam Tran et al. (2022), a qualidade e a composição dos dados influenciam diretamente a confiabilidade da avaliação, sobretudo quando poucas amostras representam determinadas classes.

Essa análise granular também deve ser interpretada como uma segmentação do classificador binário. O relatório combina cada categoria de ataque com registros benignos para calcular as métricas, mas não mede confusões entre diferentes tipos de ataque. Uma avaliação multiclasse exigiria outro alvo, outro protocolo de treinamento e uma matriz de confusão específica.

As janelas deslizantes acrescentaram contexto temporal ao modelo principal, mas o alvo pertence ao último registro da própria janela. Consequentemente, o modelo reconhece o estado corrente com apoio do histórico recente, sem estimar a ocorrência de um ataque posterior.

A avaliação prospectiva reformulou o alvo, respeitou a ordem temporal e excluiu ataques ativos ou recentes. Esse procedimento mostrou uma característica importante do UNSW-NB15: a base contém muitos fluxos, mas não informa com clareza quais registros pertencem a cada evento de ataque e reúne poucos eventos que atendem ao protocolo estrito.

Os cinco eventos remanescentes não permitem estimar a generalização para ataques novos. O conjunto continua adequado para detecção, classificação, análise de recorrência e estudos metodológicos, enquanto a previsão antecipada requer mais eventos de ataque separados e bem identificados.

2.5 Avaliação dos procedimentos

O UNSW-NB15 sustentou a avaliação temporal porque preserva `Stime` e `Ltime` e contém três sessões naturais com ambas as classes e os nove tipos de ataque.

A modularização facilitou a serialização dos modelos e a disponibilização do serviço de classificação, permitindo demonstrar o fluxo completo entre dados, modelo, API e painel. A suíte automatizada contribuiu para reduzir erros de integração e preservar o comportamento esperado dos componentes.

O protocolo final corrigiu os dois principais limites da comparação inicial. A seleção passou a determinar as entradas realmente usadas por cada modelo, e pré-processamento, seleção e janelas

passaram a respeitar os folds temporais. Identificadores únicos, purga, embargo por duração dos fluxos e hashes reduziram o risco de vazamento entre treino, validação e teste.

O teste temporal foi lido e avaliado uma única vez após o congelamento. Essa política impede que o período futuro se transforme gradualmente em conjunto de desenvolvimento. As tabelas e o artefato de serviço foram derivados dos resultados preservados, sem nova abertura do teste.

A auditoria identificou um desvio de reprodutibilidade: a execução final usou NumPy 2.5.1, Pandas 3.0.5, PyArrow 25.0.0 e scikit-learn 1.9.0, enquanto o protocolo havia registrado 2.4.2, 2.3.3, 23.0.1 e 1.8.0. O executor final também foi criado depois do congelamento. As decisões experimentais não mudaram, mas esse desvio limita a reprodução estrita e foi preservado em manifesto; repetir o teste para corrigi-lo produziria um problema metodológico maior.

A etapa prospectiva separou eventos temporalmente, aplicou purga e construiu atributos apenas com fluxos já concluídos. Esses controles preservaram a ordem temporal e mostraram que o conjunto disponível possui poucos eventos distintos e poucas partições com ambas as classes.

Uma avaliação preditiva conclusiva exigirá vários eventos de ataque completos em cada partição, com exemplos positivos e negativos para todos os horizontes. O escalonador, o seletor de atributos e qualquer ajuste de limiar deverão utilizar somente o treino, enquanto o teste permanecerá fechado até a definição final do experimento.

2.6 Conclusões e trabalhos futuros

O trabalho implementou e comparou Random Forest, Decision Tree e LSTM, além de integrar rastreamento experimental, serialização, API e painel. Na avaliação temporal principal do UNSW-NB15, o Random Forest com 30 atributos obteve o melhor resultado, com F1 de 0,9261, PR-AUC de 0,9856, precisão de 0,9010, revocação de 0,9525 e FPR de 0,0094.

A avaliação por categoria mostrou que classes raras permanecem frágeis, mesmo diante de métricas globais elevadas. Esse resultado reforça a necessidade de apresentar medidas por tipo de ataque e de evitar conclusões baseadas apenas no desempenho agregado.

O objetivo de construir um protótipo funcional foi alcançado. O artefato final recebe 43 campos brutos, aplica o pré-processamento e a seleção `top_30` e disponibiliza a inferência pela API. A pesquisa também verificou as condições necessárias para previsão antecipada ao formular rótulos futuros, estabelecer horizontes e excluir ataques ativos ou recentes.

Somente cinco eventos atenderam ao protocolo estrito, quantidade que não permitiu selecionar e avaliar um modelo prospectivo com segurança. A partir dessa constatação, o trabalho concentrou a implementação funcional na identificação do estado atual do tráfego com base em uma sequência recente.

O protocolo corrigido contribui metodologicamente em dois sentidos. Primeiro, demonstra que a seleção pode reduzir 204 atributos para 10, 20 ou 30 e ainda melhorar o F1 temporal. Segundo, mostra que um grande volume de fluxos não substitui períodos futuros isolados nem eventos de ataque distintos e bem documentados. A previsão antecipada permanece como continuidade experimental, apoiada pelos requisitos identificados nesta etapa.

A continuidade da pesquisa requer uma nova coleta controlada, na qual cada evento registre uma fase benigna, uma possível fase precursora, o início do ataque, sua atividade e a recuperação. Também serão necessárias execuções inteiramente benignas, além de eventos de ataque completos e distintos para treino, validação e teste.

Antes da coleta, o protocolo deverá definir horizontes, intervalo de purga, tamanho mínimo da amostra e métricas prioritárias. A avaliação deve incluir PR-AUC, revocação por evento, antecedência do primeiro alerta e falsos alertas por hora. Todas as transformações deverão ser ajustadas somente no treino, mantendo o teste fechado até a configuração final.

Trabalhos futuros também poderão investigar reequilíbrio de classes raras, aprendizagem sensível a custo e robustez contra evasão. Uma avaliação em tráfego universitário dependerá ainda de governança, anonimização, autorização institucional e controles de segurança.

PARTE III — RESUMO ESTENDIDO

Implementação e Avaliação de Algoritmos de Machine Learning para Previsão de Ataques Cibernéticos

Emili Vieira Tabuti* e Daniel Couto Gatti**

*Ciência da Computação, Pontifícia Universidade Católica de São Paulo

**Faculdade de Ciências Exatas e Tecnologia, Pontifícia Universidade Católica de São Paulo

E-mails: **TODO: informar e-mail institucional da estudante; TODO: informar e-mail institucional do orientador**

Resumo: Esta pesquisa implementou e avaliou algoritmos de aprendizado de máquina para apoiar alertas de segurança de redes. Random Forest, Decision Tree e LSTM foram comparados no UNSW-NB15 por três folds cronológicos expansivos e um teste futuro fechado. Pré-processamento e seleção de atributos foram ajustados somente no treino de cada fold, e as janelas de dez registros não atravessaram partições, sessões ou arquivos. O protocolo selecionou 10 atributos para Decision Tree, 20 para LSTM e 30 para Random Forest antes da única abertura do teste. Em 306.701 janelas futuras, o Random Forest alcançou F1 de 0,9261, PR-AUC de 0,9856 e revocação de 0,9525, permanecendo próximo do F1 médio de 0,9236 no desenvolvimento. O modelo vencedor foi integrado a uma API que recebe dados brutos e executa internamente pré-processamento, seleção e inferência. Uma análise prospectiva adicional definiu horizontes de 5 a 60 segundos, mas encontrou somente cinco eventos adequados após excluir ataques ativos ou recentes, número insuficiente para avaliar antecipação. O estudo entrega um protótipo funcional de detecção, uma avaliação de generalização temporal e os requisitos para continuar a pesquisa sobre previsão antecipada.

Palavras-chave: Aprendizado de Máquina. Segurança Cibernética. Detecção de Intrusões. Random Forest. LSTM.

Classificação das áreas de conhecimento: 1.03.00.00-7 — Ciência da Computação; 1.03.06.00-5 — Inteligência Artificial.

Introdução

Redes acadêmicas reúnem serviços, dispositivos e perfis de uso heterogêneos, condição que amplia a superfície de ataque e dificulta a manutenção de regras fixas. Sistemas de detecção de intrusões analisam eventos e tráfego para reconhecer comportamentos maliciosos, mas abordagens baseadas apenas em assinaturas dependem da atualização contínua de padrões conhecidos.

O aprendizado de máquina pode complementar esses mecanismos ao estimar relações entre atributos de tráfego e rótulos de segurança. A literatura apresenta árvores de decisão, métodos de ensemble e redes profundas para essa finalidade [1–4]. Contudo, diferenças entre conjuntos, atributos e protocolos experimentais dificultam comparações diretas [3]. Duplicações, sobreposições e vazamento de informação entre partições também podem elevar artificialmente as métricas [5].

Este trabalho integra uma pesquisa sobre alertas para redes acadêmicas e concentra-se no módulo de aprendizado de máquina. O estudo utilizou o UNSW-NB15, conjunto público que preserva timestamps e favorece a reprodução.

A pesquisa avaliou a possibilidade de construir rótulos futuros com os dados temporais disponíveis. Como o protocolo estrito encontrou poucos eventos adequados para essa finalidade, a

implementação funcional prosseguiu com a detecção do estado atual do tráfego a partir dos registros recentes.

Objetivos

O objetivo geral foi implementar e avaliar algoritmos de aprendizado de máquina para apoiar um sistema de alerta de segurança. Os objetivos específicos abrangeram revisão bibliográfica, seleção e treinamento dos modelos, padronização das métricas, comparação dos resultados e serialização dos artefatos.

O trabalho também avaliou se os dados disponíveis permitiam formular uma tarefa prospectiva e buscou disponibilizar o modelo selecionado por uma API integrada ao painel de visualização.

Metodologia

O estudo adotou abordagem quantitativa e experimental, sem coleta com participantes humanos. O UNSW-NB15 contém 1.584.259 registros, 43 campos brutos elegíveis e 204 atributos após o pré-processamento.

Para avaliar a viabilidade da previsão, os fluxos do UNSW-NB15 foram ordenados por arquivo e tempo. Ataques ativos foram definidos pelo intervalo inclusivo entre $Stime$ e $Ltime$, e os rótulos indicaram o início de um ataque nos 5, 15, 30 ou 60 segundos seguintes.

A etapa prospectiva excluiu instantes com ataque em andamento e calculou sessenta atributos usando somente fluxos concluídos nos 30, 60 ou 120 segundos anteriores. Um filtro adicional removeu históricos com ataques concluídos nos últimos 120 segundos, e uma purga de 180 segundos separou as partições temporais.

Como essa análise identificou poucos eventos adequados para previsão, o treinamento dos modelos prosseguiu com foco na detecção de ataques presentes nos dados.

O pré-processamento removeu duplicações exatas, tratou valores ausentes e aplicou transformação logarítmica em colunas muito assimétricas. O RobustScaler realizou o escalonamento por mediana e intervalo interquartil, enquanto as variáveis categóricas do UNSW-NB15 receberam codificação one-hot. Categorias e estatísticas foram aprendidas apenas no treino de cada fold.

Cada exemplo reuniu dez registros consecutivos. A LSTM recebeu a representação sequencial, enquanto Random Forest e Decision Tree utilizaram as janelas achatadas. O alvo correspondeu ao décimo registro, preservando nove observações anteriores como contexto.

O Random Forest utilizou cem estimadores, profundidade máxima igual a vinte e execução em dois processos paralelos. A Decision Tree também adotou profundidade máxima de vinte. A LSTM empregou 64 unidades recorrentes, uma camada densa de 32 unidades e saída sigmoide.

O UNSW-NB15 foi ordenado por tempo e dividido em três sessões naturais. Depois da purga, treino, validação e teste reuniram 1.023.187, 254.326 e 306.710 registros. O desenvolvimento usou

três folds cronológicos expansivos, sem embaralhamento e com embargo suficiente para garantir que os fluxos de treino terminassem antes do início da validação.

Cada algoritmo foi avaliado com todos os 204 atributos e com os 10, 20 e 30 primeiros de um ranking aprendido somente no treino, totalizando 36 execuções. F1 foi a métrica primária, complementada por PR-AUC, AUC-ROC, precisão, revocação e taxa de falsos positivos. O protocolo congelou Decision Tree `top_10`, LSTM `top_20` e Random Forest `top_30` antes de abrir o teste.

Os modelos finais foram ajustados em treino+validação e avaliados juntos uma única vez em 306.701 janelas do teste futuro. O Random Forest vencedor foi serializado com pré-processador, seleção e esquema bruto e disponibilizado por uma API FastAPI. Na versão auditada, a suíte automatizada aprovou 175 testes.

Resultados e discussão

No desenvolvimento temporal, o melhor F1 médio foi $0,8923 \pm 0,0235$ para Decision Tree com 10 atributos, $0,8812 \pm 0,0156$ para LSTM com 20 e $0,9236 \pm 0,0211$ para Random Forest com 30. As três seleções superaram as configurações com todos os atributos. No Random Forest, a redução de 204 para 30 atributos correspondeu a 85,3% da entrada por registro.

No teste futuro, o Random Forest alcançou F1 de 0,9261, PR-AUC de 0,9856, AUC-ROC de 0,9987, precisão de 0,9010, revocação de 0,9525 e FPR de 0,0094. A Decision Tree obteve F1 de 0,8898, e a LSTM, 0,8846. O F1 do vencedor ficou apenas 0,0025 acima da média de desenvolvimento, sem ajuste posterior ao teste.

O Random Forest apresentou o maior F1 em todas as categorias do teste, mas as classes raras permaneceram difíceis. O F1 foi 0,0301 para Worms, 0,2187 para Shellcode, 0,3054 para Analysis e 0,3187 para Backdoor. Esses valores mostram que PR-AUC e métricas por categoria são necessárias mesmo quando a separação global medida por AUC-ROC é alta.

Essas diferenças demonstram que métricas agregadas não descrevem toda a capacidade do sistema. Uma avaliação operacional deve considerar cada categoria e o custo dos erros. O protótipo serializou Random Forest e LSTM, e a API adotou o Random Forest como modelo padrão, demonstrando a integração técnica com o painel. Entretanto, não houve validação em tráfego de uma rede acadêmica real.

O teste representa uma sessão posterior, não compartilha registros com o desenvolvimento e não participa do ajuste das transformações. Por isso, o resultado constitui evidência de generalização temporal [5].

O alvo do pipeline de comparação descreve o estado no final da janela corrente. Dessa forma, o modelo identifica ataques presentes com apoio dos registros anteriores, enquanto a previsão antecipada requer horizonte explícito, atributos estritamente históricos e divisão temporal [7].

A avaliação prospectiva identificou 277 períodos de ataque em 66.484 instantes. Antes do filtro estrito, existiam entre 337 e 466 rótulos positivos, mas 85% a 97% deles continham ataques no histórico. Esses casos representavam principalmente persistência ou recorrência do mesmo período de ataque.

Após a exclusão de ataques recentes, restaram 37.311 instantes e somente cinco eventos estritos. O treino recebeu três eventos, e validação e teste receberam um evento cada. Nos horizontes de 30 e 60 segundos, as duas partições futuras não possuíam exemplos negativos.

Essa distribuição não ofereceu elementos suficientes para comparar modelos prospectivos. O UNSW-NB15 permanece adequado para detecção, classificação, recorrência e auditorias metodológicas, enquanto a previsão estrita depende de mais eventos de ataque separados e períodos benignos anteriores.

Conclusões

O trabalho implementou três modelos e um fluxo auditável de avaliação temporal, além de entregar artefato serializado, API e integração com painel. O Random Forest `top_30` apresentou o melhor equilíbrio no período futuro e foi adotado no serviço. A avaliação por categoria evidenciou fragilidade nas classes raras e confirmou que métricas globais devem ser acompanhadas por análises específicas.

A pesquisa avaliou as condições para previsão antecipada e verificou que os dados disponíveis não contêm eventos suficientes para essa avaliação. Com base nesse resultado, o protótipo foi desenvolvido para identificar o estado atual do tráfego, tarefa compatível com a estrutura das bases utilizadas.

A continuidade exige uma coleta controlada que identifique separadamente cada evento de ataque e registre períodos benignos, possíveis precursores e fases de recuperação. Eventos completos e distintos devem compor treino, validação e teste, acompanhados de execuções inteiramente benignas.

Uma auditoria identificou diferenças entre versões de dependências registradas no protocolo e usadas na execução final. As decisões permaneceram congeladas e o teste não foi repetido, mas o desvio deve acompanhar a interpretação e a reprodução do experimento. Em estudos futuros, o ambiente executável também deverá ser congelado antes da abertura do teste.

O protocolo prospectivo deverá ser definido antes do treinamento, e todas as transformações deverão ser ajustadas somente no treino. A avaliação final deve preservar um teste fechado e incluir PR-AUC, revocação por evento, antecedência do primeiro alerta e falsos alertas por hora.

Agência de Fomento

Programa Institucional de Bolsas de Iniciação Científica do Conselho Nacional de Desenvolvimento Científico e Tecnológico — PIBIC-CNPq.

Referências

- [1] BERTOLI, G. C. et al. An end-to-end framework for machine learning-based network intrusion detection system. *IEEE Access*, v. 9, p. 106790–106805, 2021. DOI: 10.1109/ACCESS.2021.3101188.
- [2] YIN, C. et al. A deep learning approach for intrusion detection using recurrent neural networks. *IEEE Access*, v. 5, p. 21954–21961, 2017. DOI: 10.1109/ACCESS.2017.2762418.
- [3] LE JEUNE, L.; GOEDEMÉ, T.; MENTENS, N. Machine learning for misuse-based network intrusion detection: overview, unified evaluation and feature choice comparison framework. *IEEE Access*, v. 9, p. 63995–64015, 2021. DOI: 10.1109/ACCESS.2021.3075066.
- [4] BERMAN, D. S. et al. A survey of deep learning methods for cyber security. *Information*, v. 10, n. 4, art. 122, 2019. DOI: 10.3390/info10040122.
- [5] TRAN, N. et al. Data curation and quality evaluation for machine learning-based cyber intrusion detection. *IEEE Access*, v. 10, p. 121900–121923, 2022. DOI: 10.1109/ACCESS.2022.3211313.
- [6] SHAO, Lisong et al. Design and implementation of a machine learning-based network intrusion detection system. In: ASIA PACIFIC CONFERENCE ON COMPUTING TECHNOLOGIES, COMMUNICATIONS AND NETWORKING, 2024, Chengdu. *Proceedings [...]*. New York: ACM, 2024. p. 137–142. DOI: 10.1145/3685767.3685790.
- [7] ANKALAKI, S. et al. Cyber attack prediction: from traditional machine learning to generative artificial intelligence. *IEEE Access*, v. 13, p. 44662–44706, 2025. DOI: 10.1109/ACCESS.2025.3547433.

REFERÊNCIAS BIBLIOGRÁFICAS

- ANKALAKI, Shilpa et al. Cyber attack prediction: from traditional machine learning to generative artificial intelligence. *IEEE Access*, v. 13, p. 44662–44706, 2025. DOI: 10.1109/ACCESS.2025.3547433.
- BERMAN, Daniel S. et al. A survey of deep learning methods for cyber security. *Information*, v. 10, n. 4, art. 122, 2019. DOI: 10.3390/info10040122.
- BERTOLI, Gustavo de Carvalho et al. An end-to-end framework for machine learning-based network intrusion detection system. *IEEE Access*, v. 9, p. 106790–106805, 2021. DOI: 10.1109/ACCESS.2021.3101188.
- CHAKIR, Oumaima et al. An empirical assessment of ensemble methods and traditional machine learning techniques for web-based attack detection in Industry 5.0. *Journal of King Saud University — Computer and Information Sciences*, v. 35, p. 103–119, 2023. DOI: 10.1016/j.jksuci.2023.02.009.
- ENNAJI, Sabrine et al. Adversarial challenges in network intrusion detection systems: research insights and future prospects. *IEEE Access*, v. 13, p. 148613–148645, 2025. DOI: 10.1109/ACCESS.2025.3600984.
- HALBOUNI, Asmaa et al. Machine learning and deep learning approaches for cybersecurity: a review. *IEEE Access*, v. 10, p. 19572–19585, 2022. DOI: 10.1109/ACCESS.2022.3151248.
- LE JEUNE, Laurens; GOEDEMÉ, Toon; MENTENS, Nele. Machine learning for misuse-based network intrusion detection: overview, unified evaluation and feature choice comparison framework. *IEEE Access*, v. 9, p. 63995–64015, 2021. DOI: 10.1109/ACCESS.2021.3075066.
- TRAN, Ngan et al. Data curation and quality evaluation for machine learning-based cyber intrusion detection. *IEEE Access*, v. 10, p. 121900–121923, 2022. DOI: 10.1109/ACCESS.2022.3211313.
- SHAO, Lisong et al. Design and implementation of a machine learning-based network intrusion detection system. In: ASIA PACIFIC CONFERENCE ON COMPUTING TECHNOLOGIES, COMMUNICATIONS AND NETWORKING, 2024, Chengdu. *Proceedings [...]*. New York: ACM, 2024. p. 137–142. DOI: 10.1145/3685767.3685790.
- YIN, Chuanlong et al. A deep learning approach for intrusion detection using recurrent neural networks. *IEEE Access*, v. 5, p. 21954–21961, 2017. DOI: 10.1109/ACCESS.2017.2762418.

ASSINATURAS

São Paulo, ____ de _____ de 2026.

Emili Vieira Tabuti
Aluna

Prof. Dr. Daniel Couto Gatti
Orientador